GPU Computing Final Project

Ettore Saggiorato
247178
Universita' di Trento
Trento, Italy
ettore.saggiorato@studenti.unitn.it

***Abstract*— This report evaluates three methods for transposing square, non-symmetric, dense matrices on CUDA-capable devices using NVIDIA's Cooperative Groups, compared against the cuBLAS library. Experiments, conducted on the University of Trento GPU Cluster, reveal that Cooperative Groups provide no significant performance benefits for this task. While cuBLAS is not always the fastest, it demonstrates greater stability in throughput, highlighting its robustness for dense matrix transposition.**

I. Introduction

The aim of this project is to develop an efficient algorithm for transposing square, dense matrices on a GPU, utilizing NVIDIA's Cooperative Groups[1]. The matrices considered in this work are dense, non-symmetric, square matrices with dimensions $\mathcal{N} \times \mathcal{N}$, where $\mathcal{N}$ is a power of 2, ranging from to 2^2 to 2^{15}.

Dense matrices are central to numerous applications in fields such as computational mathematics, numerical analysis, machine learning, data science, physics, engineering, and finance. Matrix transposition, a fundamental operation in linear algebra, is widely used across these domains for tasks such as data manipulation and optimization.

As matrix transposition is a core operation, optimizing its performance is critical for enhancing computational efficiency. This report investigates various strategies for leveraging NVIDIA's Cooperative Groups to optimize matrix transposition on the GPU, and evaluates their effectiveness in improving performance.

A. Instructions for Reproducibility

The code and instructions for reproducing the results discussed in this report are available on GitHub.

II. State of the Art

The state-of-the-art solution for transposing dense matrices on NVIDIA GPUs is provided by cuBLAS ([2]). This library supports dense matrices and vectors and offers GPU-accelerated, optimized routines for fundamental linear algebra operations. Matrix transposition is implemented through two key functions, tailored for single and double precision operations:

- **`cublasSgeam`**: The primary function for single precision matrix transposition. It allows users to specify matrix operations, including transposition, by utilizing the `CUBLAS_OP_T` operation flag.
- **`cublasDgeam`**: The corresponding function for double precision matrix transposition, analogous to cublasSgeam.

This project focuses on single precision operations.

A. Cooperative Groups

Cooperative Groups is a programming model introduced to provide more flexible and structured ways of manaigng and synchronizing threads. It allows to organize threads into hierarchical groups, enabling fine-grained control over parallelism and synhronization at different levels, like withing a warp, a block, or across the entire grid. Cooperative Groups enable thread organization by creating logical grouping of threads beyond the default block and grid structure, flexible synchronizatoin by synchronizing subsets of threads within a group instead of all theads in a block and scalability by supporting scalable algorithms by enabling cooperation between threads at multiple levels. Given the foundational nature of this work, the primary focus will be on experimenting with different algorithms and benchmarking their performance rather than introducing groundbreaking advancements in the field.

III. Contribution and Methodology

Given the capabilities of Cooperative Groups (CG), which provide APIs for granular thread management, and the relatively straightforward requirements of a dense square matrix transpose (as opposed to more complex operations like full reductions), it is anticipated that kernels developed with Cooperative Groups for this task may face performance limitations.

This project explores two primary approaches:

- **Intra-block synchronization using Cooperative Groups**: This method is expected to perform comparably to traditional implementations using `__syncthreads`, as

the synchronization scope is confined to threads within the same block.

- **Inter-block synchronization using Cooperative Groups**: This method is expected to underperform due to the significant overhead introduced by grid-wide synchronization, which is inherently more complex and time-consuming.

To evaluate these approaches, four kernels were developed:

- One reference kernel for copying operations.
- Three kernels for matrix transposition.

A. Synchronization Strategies and Kernel Descriptions

1. **Kernels with intra-block synchronization**
   Three kernels, described in Algorithm 1, Algorithm 2, and Algorithm 3, utilize the `cg::sync(cta)` function from the Cooperative Groups (CG) API to synchronize threads within a block. Here, `cta` represents the cooperative thread block group, which encompasses all threads within the block. Functionally, `cg::sync(cta)` operates similarly to the traditional `__syncthreads()` mechanism.
2. **Kernel with inter-block synchronization**
   The kernel described in Algorithm 4 demonstrates the challenges associated with inter-block synchronization for matrix transposition. This kernel employs `cudaLaunchCooperativeKernel` to synchronize across the entire grid. Due to the inherent constraints of inter-block synchronization, this kernel is limited to matrices with dimensions $\mathcal{N} \times \mathcal{N}$ with $\mathcal{N} \in [2^2, 2^5]$. The narrow range is a limitation of the current implementation due to the absence of subtitling.

```
1  set shared tile[tile_dim][tile_dim]
2  let cta ← cg::this_thread_block()
3  let x ← blockIdx.x ⋆ tile_dim + threadIdx.x
4  let y ← blockIdx.y ⋆ tile_dim + threadIdx.y
5  let width ← gridDim.x ⋆ tile_dim[1]
6  for i ← 0 to tile_dim by block_rows do:
7  |   tile[tIdx.y + i][tIdx.x] ← idata[(y + i) ∗ width + x]
8  cs::sync(cta)
9  for i ← 0 to tile_dim by block_rows do:
10 |   odata[(y + i) ∗ width + x] = tile[tIdx.y + i][tIdx.x];
```

Algorithm 1: CUDA Copy Coop

```
1  set shared tile[tile_dim][tile_dim]
2  let cta ← cg::this_thread_block()
3  for i ← 0 to tile_dim by block_rows do:
4  |   tile[tIdx.y + i][tIdx.x] ← idata[(y + i) ∗ width + x]
5  cs::sync(cta)
6  for i ← 0 to tile_dim by block_rows do:
7  |   odata[(y + i) ∗ width + x] = tile[tIdx.x][(tIdx.y + i)];
```

Algorithm 2: CUDA Transpose Coalesced Coop (CC)

```
1  set shared tile[tile_dim]                ▷ Same as Algorithm 2, but with
   [tile_dim+1]                             padding to avoid bank conflicts
```

Algorithm 3: CUDA Transpose Coalesced No Conflict (CNBCC)

```
1  set shared tile[tile_dim][tile_dim+1]
2  let grid ← cg::this_grid()
3  if (x < width and y < width):
4  |   tile[tIdx.y][tIdx.x] ← idata[y ∗ width + x]
5  grid.sync()
6  if (x < width and y < width):
7  |   odata[y ∗ width + x] ← tile[tIdx.x][tIdx.y]
```

Algorithm 4: CUDA Transpose Inter Block

IV. System Description and Experiments

The experiments were conducted on the University of Trento GPU Cluster, equipped with NVIDIA A30 GPUs. Detailed hardware and software specifications are presented in Table 1 and Table 2. Compilation of the code was performed using the nvcc compiler with the flags `-O2`, `-code=sm_80`, and `-arch=compute_80`. Hyper-threading is enabled on the cluster.

To mitigate the impact of operating system scheduling and hyper-threading noise, each experiment was repeated 10 times, with each kernel being launched 100 times per experiment. The results were averaged to ensure reliability. This setup is designed to minimize inter-program noise by maintaining consistent priority levels assigned by the operating system during each benchmark. By averaging across multiple runs, the setup also reduces variability caused by potential fluctuations in OS priorities.

The experiments were conducted on matrices with dimensions ranging from $2^2 \times 2^2$ up to $2^{15} \times 2^{15}$.

Component	Version	Docs
GPU	NVidia A30	[3]
Max. theor. bandwidth	933 GB/s	[4]
CPU	Intel XEON Gold 6238R	[5]

Table 1: Hardware Description.

[1] `x`,`y` and `width` are common in all CUDA implementations, for clarity they will be omitted in other pseudocodes.

Component	Version	Docs
OS: Rocky Linux	8.7	[6]
Scheduler: Slurm	20.11.9	[7]
CUDA	12.1.66	[8]
cuBLAS	12.1.0	[2]
C++	14	[9]

Table 2: Software Description[2].

V. Results

A. Intra-block Synchronization

The results for intra-block synchronization, shown in Figure 1, indicate no significant benefit from using Cooperative Groups for this purpose. A similar trend is observed in Figure 2 and Figure 3, confirming that intra-block synchronization with Cooperative Groups does not enhance performance for matrix transposition. The experimental results are summarized in Table 4.

Interestingly, cuBLAS performs marginally slower than the presented kernels for matrix side sizes in the range $[2^5, 2^{11}]$.

B. Inter-block Synchronization

The results in Figure 4 demonstrate that inter-block synchronization provides no performance benefits for matrix transposition. The data in Table 3 further reinforces this conclusion.

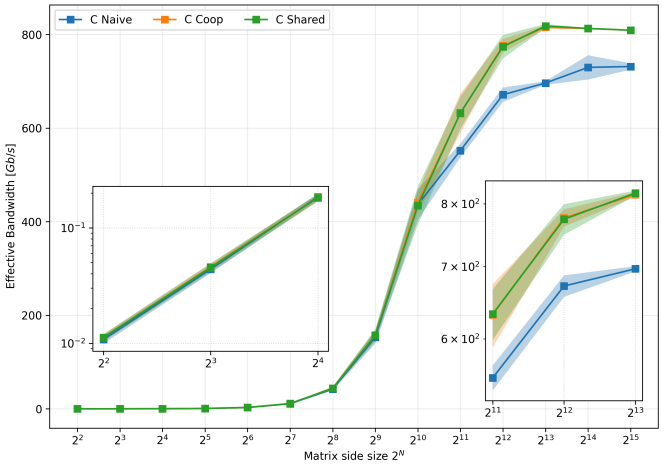

Figure 1: CUDA Matrix Copy Coop. Copy with shared memory and no bank conflicts (`C Shared`), Copy with shared memory, no bank conflicts, cooperative (`C Coop`, Algorithm 1). The two kernels using shared memory exhibit similar behavior. The average standard deviation (std) for the `C Coop` kernel is 7.12 Gbps, for `C Shared` it is 8.79Gbps, and for `C Naive` it is 8.36 Gbps.

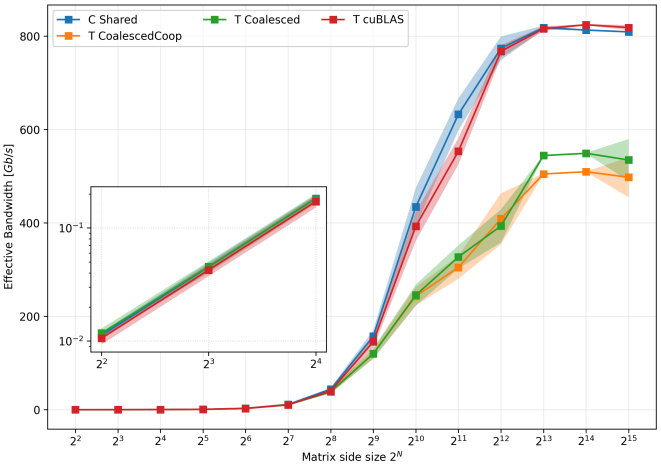

Figure 2: CUDA Matrix Transpose Coalesced Coop. Copy with shared memory (`C Shared`), Transpose with shared memory (`T Coalesced`) and Transpose Cooperative with shared memory (T CoalescedCoop, Algorithm 2). Notably, `T cublas` outperforms all other kernels. Unexpectedly, `T CoalescedCoop` performs significantly worse than `T Coalesced`, despite the only difference being the synchronization method.

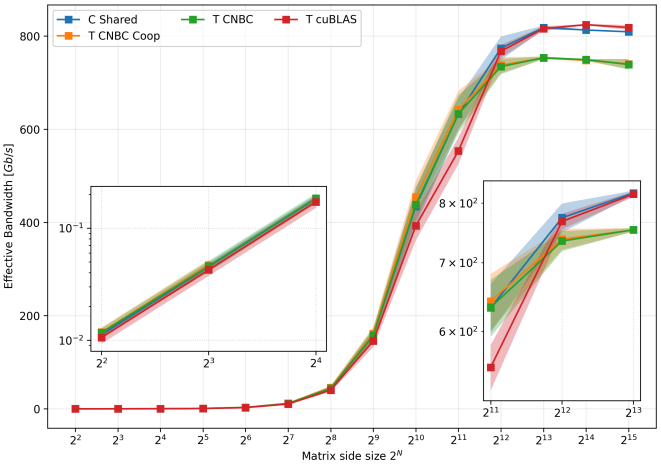

Figure 3: CUDA Matrix Transpose Coalesced No Bank Conflicts Coop. Transpose with Shared Memory, Coalesced No Bank Conflict (T CNBC) and Transpose Cooperative with Shared Memory, Coalesced No Bank Conflict (`T CNBC Coop` Algorithm 3). For $\mathcal{N} \geq 2^{12}$, `T cublas` is the best performer. However, within $2^5 \leq \mathcal{N} \leq 2^{11}$, `T CNBC Coop` outperforms the others.

[2]CMake was used for local development.

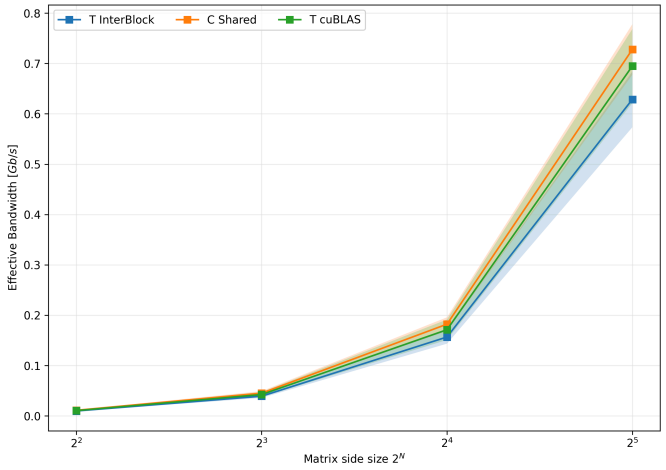

Figure 4: CUDA Transpose Inter Block. T interblock (Algorithm 4). The plot clearly shows that inter-block synchronization is not an effective choice for dense matrix transposition.

Size	InterBlock	cuBLAS
2^2	0.0098 ± 0.0008	$\mathbf{0.0106 \pm 0.0012}$
2^3	0.0392 ± 0.0033	$\mathbf{0.0425 \pm 0.005}$
2^4	0.1568 ± 0.0132	$\mathbf{0.1712 \pm 0.0202}$
2^5	0.6288 ± 0.0553	$\mathbf{0.6949 \pm 0.0734}$
avg std	0.01815	0.02495

Table 3: Inter-Block Synchronization effective bandwidths. Best results are in **bold**. All measurements are in Gbps.

Size	Coalesced	Coalesced Cooperative	Coalesced No Bank Conflict	Coalesced No Bank Conflict Cooperative	cuBLAS
2^2	0.01 ± 0	-	0.01 ± 0	0.01 ± 0	0.01 ± 0
2^3	0.05 ± 0.01	-	0.05 ± 0.01	0.05 ± 0.01	0.04 ± 0.01
2^4	0.18 ± 0.02	-	0.18 ± 0.02	-	0.17 ± 0.02
2^5	0.68 ± 0.07	0.7 ± 0.06	0.74 ± 0.07	$\mathbf{0.77 \pm 0.08}$	0.69 ± 0.07
2^6	2.67 ± 0.25	2.73 ± 0.25	2.88 ± 0.26	$\mathbf{2.98 \pm 0.33}$	2.62 ± 0.26
2^7	10.52 ± 0.96	10.82 ± 1.16	11.41 ± 1.03	$\mathbf{11.78 \pm 1.13}$	10.24 ± 1.01
2^8	38.1 ± 3.16	38.58 ± 3.79	43.95 ± 4.26	$\mathbf{45.43 \pm 4.27}$	40.22 ± 3.89
2^9	119.73 ± 11.15	120.31 ± 10.45	157.87 ± 14.52	$\mathbf{161.59 \pm 14.27}$	145.37 ± 12.97
2^{10}	245.4 ± 22.72	243.11 ± 19.54	438.17 ± 34.48	$\mathbf{454.57 \pm 34.61}$	392.82 ± 31.06
2^{11}	327.11 ± 24.79	304.7 ± 24.4	633.16 ± 40.31	$\mathbf{642.05 \pm 40.29}$	553.35 ± 28.08
2^{12}	393.42 ± 35.48	408.88 ± 54.57	734.62 ± 16.46	737.71 ± 17.19	$\mathbf{767.68 \pm 13.34}$
2^{13}	544.33 ± 1.37	504.85 ± 1.44	753.4 ± 2.82	753.16 ± 3.06	$\mathbf{816.1 \pm 5.01}$
2^{14}	549.14 ± 0.97	509.65 ± 0.74	749.55 ± 1.49	748.17 ± 2.32	$\mathbf{824.54 \pm 1.43}$
2^{15}	534.8 ± 44.6	497.79 ± 42.56	739.21 ± 11.23	740.21 ± 10.56	$\mathbf{818.46 \pm 5.42}$
avg std	10.39	14.45	9.06	9.85	7.32

Table 4: Intra-Block Synchronization effective bandwidths. **C**: coalesced, **CC**: coalesced cooperative, **CNBC** and **CNBCC**: Coalesced No Bank Conflicts (+coop). Best results are in **bold**, the best for each algorithm type are underline. All measurements are in Gbps.

VI. Conclusions

As anticipated, the application of Cooperative Groups for square dense matrix transposition does not yield significant performance advantages. This holds true for both intra-block synchronization and inter-block synchronization. While cuBLAS does not consistently outperform the custom kernels in terms of raw throughput, it exhibits a consistently lower average standard deviation. This characteristic makes cuBLAS not only competitive but also the "most stable" solution in terms of performance reliability.

These results shed light on the challenges of optimizing matrix transposition. Specifically, the straightforward nature of dense matrix transposition requires less granular control, rendering advanced thread management techniques, such as those offered by Cooperative Groups, less impactful for this task.

Future research could investigate more advanced memory management techniques, such as TMA[10], or optimize memory access patterns to improve kernel efficiency further.

References

[1] NVIDIA, [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cooperative-groups

[2] NVIDIA, "cuBLAS Library Documentation." [Online]. Available: https://docs.nvidia.com/cuda/cublas/index.html

[3] NVIDIA, "NVIDIA A30 Specs." [Online]. Available: https://www.nvidia.com/en-us/data-center/products/a30-gpu/

[4] NVIDIA, [Online]. Available: https://github.com/NVIDIA/cuda-samples/blob/master/Samples/1_Utilities/bandwidthTest/bandwidthTest.cu#L656

[5] Intel, "Intel® Xeon® Gold 6238R Processor ." [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/199345/intel-xeon-gold-6238r-processor-38-5m-cache-2-20-ghz/specifications.html

[6] R. Linux, "Rocky Linux 8.7." [Online]. Available: https://docs.rockylinux.org/release_notes/8_7/

[7] Slurm, "Slurm Documentation." [Online]. Available: https://slurm.schedmd.com/documentation.html

[8] NVIDIA, "Cuda Toolkit 12.1." [Online]. Available: https://docs.nvidia.com/cuda/archive/12.1.0/cuda-toolkit-release-notes/

[9] C. Reference, "C++ 14 Reference." [Online]. Available: https://en.cppreference.com/w/cpp/14

[10] NVIDIA, "Tensor Memory Access." [Online]. Available: https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html#tensor-memory-accelerator